

Testing Ball Detection and Tracking Scripts

This guide provides instructions on how to test and use `detection_tracker.py` and `Ball_Tracker.py` for detecting and tracking balls in videos, with specific optimizations for basketball and football.

Requirements

Required Packages

Both scripts require the following Python packages:

```
opencv-python
numpy
scipy
```

Installation

1. Install the required packages using pip:

```
pip install opencv-python numpy scipy
```

2. Download the script files to your working directory.

Using detection_tracker.py

Basic Usage

The script can be run from the command line with various options:

```
python detection_tracker.py --video [VIDEO_PATH] --mode [MODE] --output [OUTPUT_PATH]
```

Where:

- `VIDEO_PATH` is the path to your video file or camera index (e.g., `0` for webcam)
- `MODE` is either `all` (general objects) or `balls` (specific ball detection)
- `OUTPUT_PATH` is where you want to save the result video (optional)

Command-line Options

Option	Short	Description
<code>--video</code>	<code>-v</code>	Path to video file or camera index (required)

Option	Short	Description
--mode	-m	Detection mode: 'all' or 'balls' (default: all)
--output	-o	Path to save output video (optional)
--no_display		Disable display window
--max_frames		Process only first N frames
--show_mask		Show detection mask in separate window

Using Ball_Tracker.py

Basic Usage

This specialized script is designed specifically for detecting and tracking basketballs and footballs:

```
python Ball_Tracker.py VIDEO_PATH VIDEO_TYPE [options]
```

Where:

- VIDEO_PATH is the path to your input video file (required)
- VIDEO_TYPE is either basketball or football (required)

Command-line Options

Option	Short	Description
-o, --output_path		Path to save output video (optional)
-d, --display		Enable visualization window
-n, --top_n_balls		Maximum number of balls to track (default: 1)

Example Commands

```
# Track basketball in a video with visualization
python Ball_Tracker.py basketball_game.mp4 basketball --display

# Track up to 2 footballs in a video, save result
python Ball_Tracker.py football_match.mp4 football -n 2 -o
results/tracked_football.mp4

# Track basketball with default settings (no display, 1 ball)
python Ball_Tracker.py basketball_game.mp4 basketball
```

Ball Detection Parameters

Color Ranges

Both scripts use HSV color ranges to detect balls. For `Ball_Tracker.py`, these ranges are:

```
COLOR_RANGES = {
    'basketball': [
        {'lower': np.array([7, 140, 154]), 'upper': np.array([7, 152, 180])},
        {'lower': np.array([5, 100, 100]), 'upper': np.array([25, 255, 255])},
    ],
    'football': [
        {'lower': np.array([0, 0, 170]), 'upper': np.array([180, 70, 255])}, #
        {'lower': np.array([35, 20, 70]), 'upper': np.array([78, 112, 114])}, #
    ]
}
```

Ball-Specific Parameters

The `BALL_FILTER_PARAMS` dictionary contains optimized parameters for different ball types:

```
BALL_FILTER_PARAMS = {
    'basketball': {
        'min_radius': 10,
        'max_radius': 35,
        'min_circularity': 0.65,
        'max_aspect_ratio_diff': 0.5,
        'min_solidity': 0.75
    },
    'football': {
        'min_radius': 8,
        'max_radius': 17,
        'min_circularity': 0.55,
        'max_aspect_ratio_diff': 0.6,
        'min_solidity': 0.65
    }
}
```

Detection Techniques

The `Ball_Tracker.py` script uses a hybrid approach combining multiple detection methods:

1. Color-based detection

- HSV color segmentation to identify potential ball regions
- Sport-specific color ranges optimized for basketballs and footballs

2. Motion-based detection

- Background subtraction to detect moving objects
- Frame differencing to capture rapid movement

- Reflection suppression for court surfaces

3. Shape analysis

- Circularity measurement
- Aspect ratio constraints
- Solidity checks (area vs. convex hull area)

4. Tracking

- Kalman filtering with adaptive noise parameters
- Appearance modeling using color histograms
- Track management (creation, update, deletion)

Visualizations

The Ball_Tracker.py script provides real-time visualizations including:

- Bounding boxes around detected balls
- Trajectory lines showing ball movement history
- Ball ID labels
- Optional mask overlay for debugging

Customizing Ball Detection

To optimize detection for specific videos, you may need to modify:

1. **HSV Color Ranges:** Adjust the `COLOR_RANGES` values in the script to match your balls' colors and lighting conditions
2. **Size Parameters:** Modify `min_radius` and `max_radius` in `BALL_FILTER_PARAMS` based on how large the balls appear in your video
3. **Shape Parameters:** Adjust `min_circularity`, `max_aspect_ratio_diff`, and `min_solidity` to match the appearance characteristics of your balls

Troubleshooting

No Balls Detected

If balls aren't being detected:

1. Check that the balls' colors match the HSV ranges in the script
2. For basketball courts or indoor venues, adjust reflection suppression parameters
3. Try adjusting the `min_radius` and `max_radius` if balls are unusually small or large
4. Verify the video has sufficient resolution and framerate

False Detections

If too many non-ball objects are being detected:

1. Increase `min_circularity` and `min_solidity` values

2. Tighten the HSV color ranges
3. Limit `top_n_balls` to a lower number

Performance Issues

If processing is slow:

1. Reduce video resolution before processing
2. Disable the display window when saving to file
3. Consider pre-processing videos with a compression tool

Advanced Usage

Using Custom HSV Ranges

If the default color ranges don't work for your specific footage, you can modify the `COLOR_RANGES` dictionary in the script:

1. Open a sample frame in an HSV color picker tool
2. Select the ball to get its HSV values
3. Add a margin around these values to account for lighting variations

Sports-Specific Optimizations

The scripts handle different sports with specific optimizations:

- **Basketball:** Reflection suppression for shiny courts, orange color detection
- **Football:** Special handling for ground-based detections, white/brown panel detection

Controls During Execution

While `Ball_Tracker.py` is running with display enabled:

- Press `q` to quit